

Dabout

Mattéo

Oral projet IA

Partie Business :

Bonjour à tous. Quand on étudie la finance quantitative en école d'ingénieur, on se retrouve très vite face à un monument absolu : le modèle de Black-Scholes. C'est une formule mathématique d'une telle élégance et d'une telle puissance qu'elle a valu le Prix Nobel d'économie à ses créateurs.

Mais en l'étudiant en cours, il y a une hypothèse fondamentale dans cette équation qui, en tant qu'élève, m'a profondément choqué. Ce modèle repose sur le postulat que la volatilité des marchés – c'est-à-dire le niveau de risque et d'incertitude – est une constante.

Autrement dit, le modèle mathématique suppose que le marché va réagir avec le même calme plat un mardi après-midi tranquille, ou en pleine annonce d'une pandémie mondiale. C'est ce paradoxe qui m'a donné envie d'approfondir le sujet pour mon projet d'IA. Car quand on sort de la théorie pour regarder la réalité du terrain, le constat est radicalement différent.

Le vrai marché financier n'est pas un environnement mathématique parfait et continu. Il est bruité, il est asymétrique, et par-dessus tout, il est dominé par les émotions humaines. Quand le marché s'effondre, la panique s'installe. Les investisseurs se ruent sur les options de vente pour protéger leur portefeuille, et la volatilité explose. C'est ce que les professionnels appellent le « *Volatility Smile* », le sourire de volatilité.

Le problème de Black-Scholes, c'est qu'il est aveugle à cette panique. Face aux cygnes noirs et aux crises, la formule mathématique reste stoïque et sous-évalue massivement le risque réel.

C'est précisément là que réside la Proposition de Valeur Business de mon projet.

Très souvent, quand on pense à l'Intelligence Artificielle en finance, on imagine une sorte de boule de cristal qui servirait à faire de l'arbitrage. Un robot qui essaierait de prédire si une action va monter ou descendre à la milliseconde près pour générer du profit. Mais quand on parle avec des professionnels en salle de marché, on comprend que leur véritable besoin n'est pas là.

Le pire cauchemar d'un trader, ou d'une banque d'investissement, ce n'est pas de rater une opportunité de gain de quelques centimes. Son cauchemar, c'est de voir son portefeuille s'effondrer de plusieurs millions en une heure sans être protégé. Le véritable enjeu business, c'est ce qu'on appelle le Hedging : la couverture des risques.

Un trader a besoin de savoir très exactement combien d'actions il doit acheter ou vendre aujourd'hui pour être parfaitement immunisé si un krach survient demain. Avec le modèle classique, cette couverture – ce qu'on appelle le Delta – est calculée de manière théorique et rigide. Quand le marché décroche violemment, cette couverture théorique explose en vol.

J'ai donc conçu ce projet d'Intelligence Artificielle non pas comme un simple calculateur de prix, mais comme un véritable outil de « Deep Hedging dynamique ».

L'objectif de mon réseau de neurones n'est pas de renier les mathématiques, mais de les compléter. L'IA va apprendre l'historique brut du marché. Elle va ingérer les biais émotionnels,

les primes de risques cachées et les asymétries de panique que les formules classiques ignorent. L'idée que je défends aujourd'hui, c'est de passer d'une couverture de risque statique et aveugle, à un bouclier dynamique, capable d'adapter sa protection aux réalités des crises modernes.

Et pour transformer cette idée business en réalité, il a fallu relever plusieurs défis architecturaux majeurs, que je vais vous détailler tout de suite dans la deuxième partie

Partie Problèmes et Solutions :

Pour construire ce fameux bouclier dynamique dont je viens de vous parler, je suis passé à la conception de l'architecture. Et très vite, je me suis heurté à mon premier mur : la donnée.

Pour entraîner une intelligence artificielle à comprendre les options financières, il faut de l'historique de prix. Le problème, c'est que ces données valent de l'or. N'ayant pas le budget de la salle de marché de BlackRock à ma disposition, j'ai dû trouver une solution pour simuler cette réalité. J'ai donc connecté mon application à l'API de Yahoo Finance pour récupérer le vrai prix des actions sur le dernier mois. Ensuite, j'ai recréé artificiellement le prix de l'option en utilisant la formule de Black-Scholes, à laquelle j'ai injecté ce que j'appelle un "bruit de sentiment". C'est une perturbation mathématique qui simule la panique, l'euphorie, et les frictions du marché. L'objectif était clair : fournir à mon IA des données imparfaites, "sales", exactement comme dans la vraie vie, pour la forcer à apprendre ces inefficacités.

Une fois la donnée acquise, il me fallait un cerveau. C'est ici que j'ai pris une décision d'ingénierie radicale : ne faire appel à aucune "boîte noire". J'ai volontairement banni les librairies comme PyTorch, TensorFlow ou Scikit-Learn. J'ai décidé de coder l'intégralité du réseau de neurones "from scratch", à partir de zéro, en utilisant uniquement Python et l'algèbre linéaire avec NumPy. Je voulais maîtriser chaque ligne de mathématiques de mon modèle.

Le principe de mon réseau est le suivant : en entrée, 6 caractéristiques (prix, strike, temps, taux, volatilité, type d'option). Ces données traversent mes propres matrices de poids, et en sortie, on obtient un prix. Au début, l'IA propose des prix au hasard. J'ai donc dû coder manuellement l'algorithme de rétropropagation (la "Chain Rule") pour calculer l'erreur quadratique moyenne et remonter les gradients de la sortie vers l'entrée, ainsi que l'optimiseur Adam pour mettre à jour les matrices millimètre par millimètre.

En codant cela à la main, un défi s'est posé : le choix des fonctions d'activation. J'ai opté pour le standard de l'industrie, le ReLU, pour les couches cachées, car sa dérivée est ultra-rapide à calculer matriciellement. Mais pour la toute dernière couche, la sortie, j'ai codé une fonction "Softplus". Mathématiquement, c'est crucial : cela garantit de manière absolue que l'IA ne crachera jamais un prix d'option négatif.

Mais le plus grand problème restait à résoudre : Le calcul du Delta. C'est la dérivée du prix de l'option par rapport au sous-jacent. Sans une librairie comme PyTorch qui fait la dérivée automatiquement, comment faire ?

Je me suis appuyé directement sur le cours d'optimisation numérique, plus précisément sur le principe de calcul de la dérivée par différence finie, ou différence centrale. J'ai programmé l'IA pour qu'elle prévoie le prix pour un sous-jacent très légèrement augmenté (+ epsilon), puis pour un sous-jacent légèrement diminué (- epsilon). En divisant l'écart des deux prix par 2

epsilons, on obtient la pente de la tangente. Ce calcul analytique est robuste, ultra-rapide, et me donne mon fameux "Deep Delta" 100% fait maison.

À ce stade de mon projet, j'avais donc un modèle capable de comprendre le passé et d'en extraire une couverture optimale. Mais un gestionnaire de portefeuille s'intéresse rarement au passé. Son travail, c'est de survivre à demain.

J'ai donc voulu ajouter une dernière dimension à mon architecture : la projection dans le futur. Pour cela, j'ai couplé mon intelligence artificielle avec une méthode de simulation stochastique : La méthode de Monte Carlo. L'idée est d'utiliser un modèle mathématique appelé le Mouvement Brownien Géométrique pour générer des dizaines de futurs possibles pour l'action. Certains scénarios sont calmes, d'autres simulent un krach boursier violent ou une hausse vertigineuse.

Une fois ces "multivers" créés, je les donne à manger à mon IA pré-entraînée. Je lui demande : "Si le scénario catastrophe numéro 3 se réalise la semaine prochaine, quel sera le prix de l'option, et surtout, quel Delta devrai-je appliquer pour me couvrir ?"

Partie Code :

Passons maintenant au code. Je ne vais pas m'attarder sur le code de Black-Scholes ni sur le frontend en Streamlit, pour me concentrer sur le cœur de mon architecture : mon réseau de neurones codé intégralement "from scratch", c'est-à-dire de zéro, uniquement avec la librairie mathématique NumPy.

Si on regarde le premier bloc de code affiché au tableau, on a la classe CustomOptionNet. C'est le cerveau de notre IA. Au lieu d'utiliser une boîte noire comme PyTorch qui fait le travail à notre place, j'ai programmé l'initialisation de mes propres matrices de poids. On voit que les données entrent sous forme de vecteur de dimension 6 : c'est le prix de l'action, le strike, le temps, les taux, la volatilité et le type d'option. Elles traversent ensuite, par purs produits matriciels, des couches cachées de 64, puis 32 neurones, pour finalement converger vers une seule valeur de sortie : le prix.

Mais la vraie subtilité technique ici se trouve dans les fonctions d'activation. Puisque je code tout à la main, j'ai dû implémenter mathématiquement ces fonctions ainsi que leurs dérivées. J'ai choisi l'activation ReLU pour les couches cachées car sa dérivée est ultra-rapide à traiter pour le processeur. Enfin, sur la toute dernière ligne de sortie, j'ai codé et appliqué une fonction Softplus. C'est une sécurité mathématique qui garantit que la prédiction finale sera toujours strictement positive, car une option ne peut pas avoir un prix négatif.

Ensuite, nous avons la classe AIOptionPricer qui gère l'orchestration. Dans le constructeur, je prépare mes outils faits maison : un StandardScaler que j'ai codé pour normaliser nos données financières qui ont des échelles très différentes, et les paramètres de mon optimiseur Adam avec un taux d'apprentissage de 0.01.

Si l'on regarde la méthode train, c'est ici que la théorie mathématique prend vie. Après avoir normalisé les matrices de données, on entre dans la boucle d'apprentissage. Chaque itération se fait en 4 étapes mathématiques strictes :

1. Le Forward pass : Le modèle effectue ses produits matriciels pour tenter une prédiction, et je stocke les résultats intermédiaires dans un "cache".

2. Le calcul de la Loss : Je calcule manuellement l'erreur quadratique moyenne (MSE) pour mesurer l'écart avec la réalité.
3. Le Backward pass : C'est l'étape la plus complexe. Plutôt que de lancer une fonction automatique, j'ai codé la règle de la chaîne, la "Chain Rule". Je remonte l'architecture à l'envers, en multipliant les dérivées locales pour trouver le gradient d'erreur de chaque poids du modèle.
4. Le Step : Mon algorithme Adam met à jour chaque matrice de poids en fonction de ces gradients.

Une fois entraîné, le modèle peut faire des prédictions instantanées. L'IA n'a plus besoin de calculer de gradients, un simple *Forward pass* suffit, ce qui accélère énormément le calcul.

Mais là où j'ai dû adapter mon ingénierie, c'est dans la méthode `compute_deep_delta`. Comment calculer le Delta, c'est-à-dire la dérivée du prix de l'option par rapport au sous-jacent, sans utiliser de librairie de différentiation automatique ? Je me suis directement appuyé sur nos cours d'optimisation numérique. J'ai utilisé la méthode de la différence centrale.

Concrètement, je demande à mon réseau de prédire le prix pour le sous-jacent augmenté d'un minuscule epsilon, puis diminué de ce même epsilon. Je divise cet écart de prix par deux epsilon pour obtenir la pente de la tangente. Cette technique est robuste, mathématiquement saine, et surtout, je n'oublie pas de re-multiplier le résultat par l'écart-type originel de mon `StandardScaler` maison pour annuler la normalisation et obtenir le véritable Delta en euros.

Enfin, pour utiliser ce Deep Delta en conditions réelles, j'ai implémenté le générateur de scénarios : `generate_future_paths`. C'est une implémentation pure du Mouvement Brownien Géométrique. J'initialise une matrice de chemins remplie de zéros. Puis, je boucle sur les jours futurs. À chaque itération, je génère Z , un choc aléatoire tiré d'une loi normale standard. Le prix de demain `paths[t]` est calculé en prenant le prix d'aujourd'hui, auquel j'applique une dérive – le μ – et un choc de volatilité σ multiplié par la racine du pas de temps dt et mon choc aléatoire Z .

Cela me génère mes multivers financiers, que je peux ensuite injecter dans mon réseau PyTorch pour qu'il me prédise mes prix et mes Deltas futurs.

Partie Finale (Exemple + Conclusion) :

Pour conclure cette présentation, j'aimerais revenir sur ce qui a rendu ce projet particulièrement intéressant à réaliser. Ce qui m'a le plus passionné, c'est de réussir à faire le pont entre la théorie mathématique financière pure – comme le modèle de Black-Scholes qui peut sembler très abstrait sur le papier – et la puissance concrète du Deep Learning. Voir concrètement qu'un réseau de neurones n'est pas juste une "boîte noire" qui devine un prix, mais que l'on peut utiliser la différentiation automatique pour en extraire une véritable dérivée mathématique applicable sur les marchés, c'est extrêmement formateur. Cela m'a vraiment permis de comprendre par la pratique l'importance vitale d'adapter la technologie aux vrais besoins métiers, en l'occurrence la couverture de risques, le Hedging.

Bien sûr, ce modèle de "Deep Hedging" n'est qu'un prototype fonctionnel et il reste de nombreuses pistes d'amélioration pour le rapprocher encore plus de la réalité d'une véritable salle de marché.

Tout d'abord, au niveau de l'intelligence artificielle elle-même. Actuellement, le modèle apprend les prix de manière purement empirique. Il serait très intéressant de s'inspirer des *Physics-Informed Neural Networks*. L'idée serait d'ajouter des contraintes financières strictes directement dans la fonction d'erreur (la *Loss Function*). On pourrait ainsi interdire formellement à l'IA de proposer des opportunités d'arbitrage absurdes, comme un Call dont le prix serait inférieur à sa valeur intrinsèque. Cela rendrait le modèle non seulement performant, mais mathématiquement irréprochable.

Ensuite, au niveau de l'infrastructure et de la donnée pure. Pour le moment, l'application tourne localement et s'entraîne sur un historique simulé à partir de Yahoo Finance. Pour passer véritablement à l'échelle industrielle, la prochaine étape logique serait de se brancher sur un véritable flux de données d'options en temps réel, via des API professionnelles comme Bloomberg ou Reuters. Il faudrait alors déployer ce dashboard et une architecture style PyTorch dans le Cloud, par exemple sur AWS ou Google Cloud, afin d'entraîner le modèle en continu pour qu'il réajuste sa couverture minute par minute au gré des secousses du marché.

Je vous remercie pour votre attention tout au long de cette présentation, et je suis maintenant à votre entière disposition si vous avez des questions sur le code, l'architecture ou mes choix techniques.